

ixsurface

Interactive 3D surface plots for visualizing multi-factor interactions from fitted models.

Instead of examining combinatorial pairwise interaction plots, **ixsurface** maps factor combinations to response surfaces and uses **surface crossings** as geometric indicators of interaction effects. Where surfaces cross, the effect of one factor depends on the level of another; the definition of an interaction.

Installation

```
# From GitHub
devtools::install_github("cjbrant/ixsurface")

# Or from local source
install.packages("ixsurface/", repos = NULL, type = "source")
```

Requirements: R $\geq$ 4.0, plotly $\geq$ 4.10.0

Quick Start

```
library(ixsurface)

# Simulate a mixed design: 2 continuous + 1 categorical factor
dat = sim_factorial(n = 300, design = "mixed", seed = 42)
fit = lm(y ~ temp * pressure * catalyst, data = dat)

# 3D surface plot with one surface per catalyst level
interaction_surface(fit, x = "temp", y = "pressure", facet_by = "catalyst")
```

This produces an interactive plotly widget with three colored surfaces. Rotate, zoom, and hover to inspect predicted values. Where surfaces cross, the temp-pressure relationship differs across catalyst levels — a three-way interaction.

How It Works

1. **Prediction grid** — A regular grid is constructed over the focal variables (x, y). Non-focal, non-facet variables are held at their median (continuous) or mode (categorical).
2. **Surface generation** — For each level of `facet_by`, predicted responses are computed over the grid and rendered as a 3D surface.

3. **Crossing detection** — Grid cells where the z-difference between surface pairs changes sign are flagged as crossings. An adaptive tolerance (2% of z-range) filters noise.
4. **Geometric interpretation:**
 - **Parallel surfaces** = no interaction between `facet_by` and the focal variables
 - **Crossing surfaces** = interaction present
 - **Twisted/warped surfaces** = higher-order or nonlinear interaction

Functions

Core Visualization

Function	Description
<code>interaction_surface()</code>	Main function. Generates interactive 3D surface plot from a fitted model with optional crossings, contour projection, and observed data overlay.
<code>plot_crossings()</code>	Standalone crossings-only 3D scatter plot. Isolates where interaction effects are strongest.
<code>interaction_surface_grid()</code>	Generates all $C(k, 2)$ pairwise interaction surface plots for a model.

Analysis

Function	Description
<code>find_crossings()</code>	Returns a data frame of approximate crossing locations between surfaces.
<code>make_prediction_grid()</code>	Builds the prediction grid over focal variables, with automatic binning for continuous <code>facet_by</code> variables.

Utilities

Function	Description
<code>sim_factorial()</code>	Simulates factorial experiment data with known interaction structure. Supports "mixed", "continuous", and "categorical" designs.
<code>bin_continuous()</code>	Bins a continuous variable into discrete levels using quantile, equal-width, or pretty breakpoints.

Supported Models

Any model with a `predict()` method works:

- `lm()` / `aov()` — linear models
- `glm()` — generalized linear models (predictions on response scale, e.g., probabilities for logistic)
- `gam()` — generalized additive models
- Mixed-effects models with a `predict(newdata = ...)` method

For GLM family models, predictions are automatically returned on the response scale via `predict(..., type = "response")`.

Design Types

```
# Mixed: 2 continuous + 1 categorical
dat = sim_factorial(design = "mixed", seed = 1)
# -> temp (continuous), pressure (continuous), catalyst (factor: A, B, C)

# Continuous: 3 continuous factors
dat = sim_factorial(design = "continuous", seed = 1)
# -> temp, pressure, speed (all continuous)

# Categorical: 3 categorical factors
dat = sim_factorial(design = "categorical", seed = 1)
# -> catalyst (A, B, C), operator (Op1, Op2), shift (Day, Night)
```

When a continuous variable is used as `facet_by`, it is automatically binned into discrete groups. Control this with `n_bins` and `bin_method`.

Examples

All features enabled

```
interaction_surface(  
  fit, x = "temp", y = "pressure", facet_by = "catalyst",  
  show_points = TRUE, show_crossings = TRUE, show_contour = TRUE,  
  alpha = 0.5,  
  labs = list(x = "Temperature (C)", y = "Pressure (psi)", z = "Yield (%)")  
)
```

Crossings-only view

```
plot_crossings(fit, "temp", "pressure", "catalyst",  
  labs = list(x = "Temperature (C)", y = "Pressure (psi)", z = "Yield"))
```

GLM (logistic regression)

```
dat$success = rbinom(nrow(dat), 1, plogis((dat$y - 50) / 5))  
gfit = glm(success ~ temp * pressure * catalyst, data = dat, family = binomial)  
  
interaction_surface(gfit, x = "temp", y = "pressure", facet_by = "catalyst",  
  labs = list(z = "P(success)"))
```

Pairwise exploration

```
plots = interaction_surface_grid(fit, n = 20)  
plots$temp__pressure  
plots$temp__catalyst
```

Programmatic crossing analysis

```
cx = find_crossings(fit, "temp", "pressure", "catalyst")  
# cx is a data.frame: cx, cy, cz, pair_label  
table(cx$pair_label)
```

License

MIT